

Seguridad Informática.

Practica 2 – Algoritmo RSA

Usando software / herramienta digital (Python)

```

RSA.py X modulo.py X
RSA.py > ...
1 """
2 Practica 2 - Algoritmo RSA
3
4 """
5
6 import math
7
8 # ----- Datos de entrada -----
9 p = 67
10 q = 83
11
12 # ----- Paso 1: Calcular n -----
13 n = p * q
14 print(f"Paso 1: n = p * q = {p} * {q} = {n}")
15
16 # ----- Paso 2: Calcular phi(n) -----
17 phi = (p - 1) * (q - 1)
18 print(f"Paso 2: phi(n) = (p-1)*(q-1) = {p-1} * {q-1} = {phi}")
19
20 # ----- Paso 3: Elegir e (el 4to candidato valido) -----
21 candidatos = []
22 e_actual = 2
23 while len(candidatos) < 4:
24     if math.gcd(e_actual, phi) == 1:
25         candidatos.append(e_actual)
26     e_actual += 1
27
28 print(f"Paso 3: candidatos validos encontrados (en orden): {candidatos}")
29 e = candidatos[3] # el cuarto candidato (indice 3)
30 print(f"e elegido = 4to candidato = {e}")
31
32 # ----- Paso 4: Calcular d (inverso multiplicativo de e mod phi(n)) -----
33 d = pow(e, -1, phi)
34 print(f"Paso 4: d (inverso de e mod phi(n)) = {d}")
35 print(f"Comprobacion: (e * d) mod phi(n) = {(e * d) % phi} (debe ser 1)")
36
37 # ----- Paso 5: Descifrar el mensaje -----
38 mensaje_cifrado = [1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597]
39

```

1.- Esta primera parte del script inicia importando la librería `math`, necesaria para usar la función `math.gcd()` más adelante. Se definen los datos de entrada del problema, $p = 67$ y $q = 83$, tal como indica el enunciado de la práctica.

En el **Paso 1** se calcula n multiplicando directamente $p \times q$, obteniendo $n = 5561$; este valor será el módulo que se usará tanto para cifrar como para descifrar.

En el **Paso 2** se calcula $\phi(n)$ aplicando la fórmula de Euler para dos primos, $(p-1)(q-1) = 66 \times 82 = 5412$. Este valor es fundamental porque tanto e como d deben cumplir relaciones matemáticas respecto a él.

Después inicia el **Paso 3**, donde se declara una lista vacía `candidatos = []` y una variable contadora `e_actual = 2` (se empieza en 2 porque e debe ser mayor que 1). Se abre un ciclo `while len(candidatos) < 4:` que se repetirá hasta encontrar exactamente cuatro valores de e válidos. Dentro del ciclo, `math.gcd(e_actual, phi)` calcula el máximo común divisor entre el candidato actual y $\phi(n)$; si el resultado es 1 (son coprimos), el candidato se agrega a la lista con `candidatos.append(e_actual)`. Al final de cada iteración, `e_actual += 1` incrementa el contador para probar el siguiente número.

Por último se define `mensaje_cifrado`, la lista de los 10 números que representan el criptograma a descifrar en el Paso 5, tomados directamente del enunciado de la práctica.

```
RSA.py X modulo.py X
RSA.py > ...
19
20 # ----- Paso 3: Elegir e (el 4to candidato valido) -----
21 candidatos = []
22 e_actual = 2
23 while len(candidatos) < 4:
24     if math.gcd(e_actual, phi) == 1:
25         candidatos.append(e_actual)
26     e_actual += 1
27
28 print(f"Paso 3: candidatos validos encontrados (en orden): {candidatos}")
29 e = candidatos[3] # el cuarto candidato (indice 3)
30 print(f"    e elegido = 4to candidato = {e}")
31
32 # ----- Paso 4: Calcular d (inverso multiplicativo de e mod phi(n)) -----
33 d = pow(e, -1, phi)
34 print(f"Paso 4: d (inverso de e mod phi(n)) = {d}")
35 print(f"    Comprobacion: (e * d) mod phi(n) = {(e * d) % phi} (debe ser 1)")
36
37 # ----- Paso 5: Descifrar el mensaje -----
38 mensaje_cifrado = [1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597]
39
40 print("\nPaso 5: Descifrando mensaje...")
41 numeros_descifrados = []
42 for c in mensaje_cifrado:
43     m = pow(c, d, n) # exponenciacion modular rapida integrada en Python
44     numeros_descifrados.append(m)
45     print(f"    {c}^{d} mod {n} = {m} -> caracter: '{chr(m)}'")
46
47 texto = "".join(chr(num) for num in numeros_descifrados)
48 print(f"\nMensaje descifrado: {texto}")
49
50 # ----- Resumen de claves -----
51 print("\nResultados finales:")
52 print(f"Clave publica: (e={e}, n={n})")
53 print(f"Clave privada: (d={d}, n={n})")
54 print(f"Mensaje descifrado: {texto}")
```

2.- Esta segunda parte del código continúa el ciclo `while` del Paso 3 explicado en la Figura 1, mostrando cómo se recorren los candidatos de `e` usando `math.gcd()` como condición de validación dentro de un `if`. Una vez que la lista `candidatos` contiene los cuatro valores requeridos, se imprime la lista completa con un `f-string` (línea 28) para dejar evidencia de todos los números probados, y se selecciona el cuarto elemento con `candidatos[3]` (recordando que en Python los índices inician en 0, por lo que el índice 3 corresponde al cuarto valor). Este se guarda en la variable `e`, quedando `e = 17`.

En el **Paso 4** se calcula `d` usando `d = pow(e, -1, phi)`. Esta es una función nativa de Python (disponible desde la versión 3.8) que calcula directamente el **inverso multiplicativo modular**: internamente aplica el algoritmo extendido de Euclides para encontrar el número `d` tal que $(e \times d) \bmod \phi(n) = 1$, sin que el programador tenga que implementarlo manualmente. El resultado obtenido es `d = 4457`. Inmediatamente después se imprime una comprobación calculando $(e * d) \% \phi$, donde el operador `%` obtiene el residuo de la división; si el resultado es 1, confirma que `d` fue calculado correctamente.

En el **Paso 5** se recorre cada número `c` de `mensaje_cifrado` con un ciclo `for`, y por cada uno se aplica `pow(c, d, n)`. Aquí `pow()` se usa con **tres argumentos** (base, exponente, módulo), lo que activa el algoritmo de **exponenciación modular rápida** (exponenciación binaria o "cuadrado y multiplica")

integrado en Python; esto permite calcular c elevado a la potencia d , módulo n , de forma eficiente aunque d sea un número grande (4457), evitando calcular c^{4457} completo antes de aplicar el módulo. Cada resultado m corresponde al código ASCII de un carácter del mensaje original, el cual se convierte a letra con `chr(m)` y se guarda en la lista `numeros_descifrados` mediante `.append()`.

Finalmente, `"".join(chr(num) for num in numeros_descifrados)` concatena todos los caracteres obtenidos en un solo string (el operador `join` une los elementos de una lista/generador usando como separador el string vacío `""`), reconstruyendo así el mensaje descifrado completo. El script cierra imprimiendo un resumen con la clave pública (e, n) y la clave privada (d, n) .

```

Paso 1:  $n = p * q = 67 * 83 = 5561$ 
Paso 2:  $\phi(n) = (p-1)*(q-1) = 66 * 82 = 5412$ 
Paso 3: candidatos validos encontrados (en orden): [5, 7, 13, 17]
        e elegido = 4to candidato = 17
Paso 4: d (inverso de e mod  $\phi(n)$ ) = 4457
        Comprobacion:  $(e * d) \bmod \phi(n) = 1$  (debe ser 1)

Paso 5: Descifrando mensaje...
1058^4457 mod 5561 = 72 -> caracter: 'H'
2597^4457 mod 5561 = 79 -> caracter: 'O'
4955^4457 mod 5561 = 76 -> caracter: 'L'
4201^4457 mod 5561 = 65 -> caracter: 'A'
3162^4457 mod 5561 = 32 -> caracter: ' '
4343^4457 mod 5561 = 77 -> caracter: 'M'
2754^4457 mod 5561 = 85 -> caracter: 'U'
2497^4457 mod 5561 = 78 -> caracter: 'N'
336^4457 mod 5561 = 68 -> caracter: 'D'
2597^4457 mod 5561 = 79 -> caracter: 'O'

Mensaje descifrado: HOLA MUNDO

Resultados finales:
Clave publica: (e=17, n=5561)
Clave privada: (d=4457, n=5561)
Mensaje descifrado: HOLA MUNDO

```

3.- Esta captura muestra la consola con la salida completa del programa al ejecutarse, permitiendo verificar que cada paso del algoritmo produjo el resultado esperado.

Se observa que **Paso 1** confirma $n = 67 \times 83 = 5561$, y **Paso 2** confirma $\phi(n) = 66 \times 82 = 5412$, coincidiendo con los cálculos manuales.

En **Paso 3** se listan los candidatos válidos en el orden en que fueron encontrados por el ciclo while: [5, 7, 13, 17]. Esta lista es la evidencia directa de que se probaron los números en secuencia y que, efectivamente, 17 es el cuarto valor que cumple la condición $\gcd(e, \phi(n)) = 1$, justificando por qué se eligió como clave pública.

En **Paso 4** se muestra el valor calculado $d = 4457$ mediante $\text{pow}(e, -1, \phi)$, junto con la línea de comprobación que confirma $(e \times d) \bmod \phi(n) = 1$, validando matemáticamente que d es el inverso correcto de e .

En **Paso 5** se despliega, línea por línea, el descifrado de cada uno de los 10 números del mensaje cifrado: se muestra la operación realizada (por ejemplo, $1058^{4457} \bmod 5561 = 72$) y el carácter ASCII correspondiente entre comillas simples (por ejemplo, 'H'). Al concatenar los 10 caracteres obtenidos se reconstruye el mensaje original: **HOLA MUNDO**.

Por último, la sección "Resultados finales" resume ambas claves generadas —la clave pública ($e=17$, $n=5561$), que serviría para cifrar mensajes, y la clave privada ($d=4457$, $n=5561$), usada para descifrarlos— junto con el mensaje descifrado como confirmación final del ejercicio

Iván José Rodríguez Maldonado

05/09/2026

Práctica 2 - Algoritmo RSA

Paso 1 - calcular n

$p = 67$

$q = 83$

$n = p * q$

$n = (67) * (83)$

$n = 5561$

Paso 2: Calcular $\phi(n)$

$\phi(n) = (p-1) * (q-1)$

$\phi(n) = (67-1) * (83-1)$

$\phi(n) = 5412$

Paso 3: Elegir e (clave pública)

Factorizar

$5412 = 2 * 2 * 3 * 3 * 3 * 3 * 3$

$2 * 7 * 0 * 6 * 2$

$1 * 3 * 5 * 3 * 3$

$4 * 5 * 1 * 1 * 1$

$4 * 4 * 4 * 1$

1

$2 \times$

$3 \times$

$4 \times$

$5 \checkmark$

$6 \times$

$7 \checkmark$

$8 \times$

$9 \times$

$10 \times$

$11 \times$

$12 \times$

$13 \checkmark$

$14 \times$

$15 \times$

$16 \times$

$17 \checkmark$

$18 \times$

$19 \times$

$e = 17$

Paso 4: Calcular d (Clave privada)

$$5412 = 17 \times 318 + 6 \rightarrow 6 = 5412 - 17 \times 318$$

$$17 = 6 \times 2 + 5 \rightarrow 5 = 17 - 6 \times 2$$

$$6 = 5 \times 1 + 1 \rightarrow 1 = 6 - 5 \times 1$$

$$1 = 6 - 5 \times 1$$

$$1 = 6 - (17 - 6 \times 2)$$

$$1 = 6 \times 3 - 17$$

$$1 = (5412 - 17 \times 318) \times 3 - 17$$

$$1 = (5412)3 - 17(955)$$

$$17(-955) \equiv 1 \pmod{5412}$$

$$d = -955 + 5412 = 4457$$

Paso 5: Decifrar el mensaje con la clave privada

$$1058 \quad 2597 \quad 4955 \quad 8201 \quad 3162 \quad 4343 \quad 2734 \quad 2497 \quad 336$$

$$2597$$

$$4457 \text{ a binario} = 1000101101001$$

										ASCII	Character
1	0	5	8	*	4457	mod	5564	=	72		"H"
2	3	9	7	^	4457	mod	5564	=	79		"O"
4	9	3	5	^	4457	mod	5564	=	76		"L"
4	2	0	1	^	4457	mod	5564	=	65		"A"
3	1	6	2	^	4457	mod	5564	=	32		" "
4	3	4	3	^	4457	mod	5564	=	77		"M"
2	7	5	4	^	4457	mod	5564	=	85		"U"
2	4	4	7	^	4457	mod	5564	=	78		"N"
3	3	6		^	4457	mod	5564	=	68		"D"
2	5	9	7	^	4457	mod	5564	=	79		"O"